SOA & Microservices - CCS3341

Module Code / Title:	CCS3341 SOA & Microservices
Assessment Component:	Coursework
Weighting:	40%
Handed out	Friday, 6th March 2026
Due Date:	Friday, 27th March 2026 at 1pm
Learning Outcomes:	ILO1: Describe SOA to structure web-based system ILO2: Explain WS* services ILO3: Apply REST architecture ILO4: Implement microservices in cloud environments.

Expected Deliverables:	• **Design Artifacts:** SOA design doc, WSDL files, UDDI entries, governance policy.
	• **Source Code:**
	  ○ CatalogService (Java SOAP WAR)
	  ○ OrdersService (Spring Boot or Node.js REST)
	  ○ BPEL process definitions
	  ○ Integration code/config
	• **Configuration:** sun-jaxws.xml, web.xml, Spring Security/OAuth2, Integration exchanges/queues, BPEL deployment descriptors.
	• **Test Suites:** SOAP UI project; Postman or curl scripts; BPEL engine console logs; Integration queue status screenshots.
	• **Reflective Report :** Trade-off analysis
	• **Video demonstration**

OVERVIEW

GlobalBooks Inc., a rapidly expanding international e-commerce organization, is undertaking a strategic migration from a tightly coupled monolithic application to a distributed Service - Oriented Architecture. The organization requires independent, scalable, secure, and governable services that can operate reliably under real-world conditions.

Students will assume the roles of:

- **Solution Architect** – defining service boundaries and architectural principles
- **Service Developer** – implementing SOAP and REST services
- **Integration Specialist** – configuring messaging and orchestration
- **Security Engineer** – applying WS-Security and OAuth2
- **Governance Consultant** – defining versioning, SLA, and lifecycle strategies

The coursework requires students to design, implement, integrate, secure, test, and govern four independent services:

- CatalogService (SOAP)
- OrdersService (REST)
- PaymentsService (Asynchronous Messaging)
- ShippingService (Asynchronous Messaging)

Students must produce design documentation, working service implementations, orchestration definitions, security configurations, governance policies, testing evidence, and participate in a viva demonstration.

The coursework duration is 3 weeks, simulating a real-world sprint-based transformation project.

SCENARIO

GlobalBooks Inc. has grown into a global online bookstore serving millions of customers across North America, Europe, and Asia. The company's current system is a legacy Java-based monolithic application where all business functions - catalog browsing, order placement, payment processing, and shipment coordination are tightly integrated into a single codebase and database.

Current Problems

- Tight coupling between components
- Single shared database
- Difficult deployments
- Full regression testing for minor updates
- Scalability bottlenecks during promotional campaigns
- Limited technology flexibility
- High operational risk during peak sales events

During major events such as holiday promotions and global author launches, the platform struggles to handle increased traffic, resulting in performance degradation and occasional outages.

To modernize the platform, the CTO has approved a transition to a Service-Oriented Architecture with the following requirements:

Required Services

1. **CatalogService**
    - Provides book lookup and price information
    - Must expose SOAP endpoints for legacy partners
2. **OrdersService**
    - Handles order creation and retrieval
    - Must expose REST APIs for modern clients
3. **PaymentsService**
    - Processes payment confirmation
    - Integrated asynchronously via messaging
4. **ShippingService**
    - Handles shipment coordination
    - Triggered via asynchronous messaging

Technical Requirements

- SOAP (WSDL-based contracts)
- REST (JSON-based APIs)
- UDDI-based service discovery
- RabbitMQ as ESB for asynchronous messaging
- BPEL engine for orchestration of the “PlaceOrder” workflow
- WS-Security for SOAP endpoints
- OAuth2 for REST endpoints
- QoS mechanisms for reliability
- Governance framework (versioning, SLA, deprecation)

Students must demonstrate the working system under normal and failure scenarios during the viva.

QUESTIONS

Answer ALL questions clearly and provide supporting diagrams, configurations, and code snippets where required.

Section A – Architecture & Service Design

1. Explain which SOA design principles you applied when decomposing the monolithic system into independent services. (10 Marks)
2. Discuss one key benefit and one primary challenge of your chosen decomposition approach. (5 Marks)
3. Provide a WSDL excerpt for the CatalogService including:
   - Operations
   - Message types
   - Binding
   - Service endpoint (6 Marks)
4. Draft the UDDI registry metadata entry required for client discovery of the CatalogService. (4 Marks)

Section B – Service Implementation & Testing

5. Describe in detail how you implemented the CatalogService SOAP endpoint in Java, including:
   - Service annotations
   - sun-jaxws.xml configuration
   - web.xml configuration (10 Marks)
6. Explain how you tested the SOAP service using SOAP UI, including:
   - Test cases
   - Assertions
   - Negative test scenarios (5 Marks)
7. Design the OrdersService REST API. Include:
   - Endpoint definitions (POST /orders, GET /orders/{id})
   - Sample JSON request and response
   - JSON Schema for order creation (15 Marks)
8. Outline the “PlaceOrder” BPEL orchestration process, including:
   - Receive activity
   - Loop for price lookup
   - Invocation of OrdersService
   - Reply to client (10 Marks)
9. Explain how you deployed and tested the BPEL process on a BPEL engine (e.g., Apache ODE). (5 Marks)

Section C – Integration & Messaging

10. Explain how you integrated the PaymentsService and ShippingService using RabbitMQ, including:

- Exchange definitions
- Queue configurations
- Producer and consumer roles (7 Marks)

11. Describe your error-handling strategy, including:

- Retry mechanisms
- Dead-letter queues
- Failure routing (3 Marks)

Section D – Security, QoS & Governance

12. Detail the WS-Security configuration for the CatalogService (UsernameToken or X.509 approach). (4 Marks)
13. Describe the OAuth2 configuration for securing the OrdersService REST API. (4 Marks)
14. Explain one Quality of Service (QoS) mechanism configured for reliable messaging. (2 Marks)
15. Draft the governance policy including:

- Versioning strategy (URL and namespace conventions)
- SLA targets (availability and response time)
- Deprecation strategy (notice period and sunset process) (10 Marks)

DELIVERABLES

Students must submit the following:

1. Design Artifacts

- SOA Architecture Design Document
- Service decomposition diagram
- WSDL files
- REST API specification
- JSON Schema definitions
- UDDI registry metadata
- Governance policy document

2. Source Code

- CatalogService (Java SOAP WAR)
- OrdersService (Spring Boot or Node.js REST)
- PaymentsService integration code
- ShippingService integration code
- BPEL process definition files

3. Configuration Files

- sun-jaxws.xml
- web.xml
- WS-Security configuration
- OAuth2 configuration
- RabbitMQ exchange and queue definitions
- BPEL deployment descriptors

4. Testing Evidence

- SOAP UI project file
- Postman collection or curl scripts
- BPEL engine deployment logs
- RabbitMQ dashboard screenshots
- Load testing or failure simulation screenshots

5. Reflective Report

A critical reflection including:

- Architectural trade-offs
- SOA vs Microservices considerations
- Challenges faced during integration
- Security and governance observations
- Scalability analysis

6. Video Demonstration Requirement

Students must record and upload a **video demonstration (15 minutes)** showing the execution of the implemented system. The video must clearly demonstrate:

- SOAP service execution for **CatalogService**
- REST API functionality for **OrdersService**
- BPEL orchestration workflow for **PlaceOrder**
- Messaging interaction between **PaymentsService** and **ShippingService**
- Security mechanisms (**WS-Security** and **OAuth2**)
- Error-handling or failure scenarios

The video should include a **clear explanation of the architecture and system components**. Students must provide a **link to the recorded video (e.g., YouTube, OneDrive, or Google Drive)** within the submitted documentation. The link must be accessible to the lecturer for evaluation and verification during the viva.

END OF THE PAPER